Reviewer Pack — Minimal Rank of Geometric Quantization over Boolean Algebras…

Entry b7c1e9607c6e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 02:46:12 UTC

Minimal Rank of Geometric Quantization over Boolean Algebras vs Communication Protocol Efficiency

Entry ID: `b7c1e9607c6e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 02:46:12 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization **Field B** (complexity object): Communication Complexity

Statement:

> {'sentence_1': 'The minimal rank of the geometric quantization matrix for a given boolean algebra is proportional to the communication protocol efficiency required to solve the associated problem.', 'sentence_2': 'For any boolean algebra with n variables, there exists a constant c such that the communication protocol efficiency E[CP] satisfies E[CP] = O(c * log(n)) where CP is measured in terms of bits or rounds.', 'sentence_3': 'This proportionality holds for problems expressible as communication games, like the Disjointness problem.'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Geometric quantization offers a framework to study algebraic structures and their geometric properties, which could reveal non-trivial relationships between abstract algebra and computational complexity.', 'sentence_2': 'By mapping boolean algebras to geometric quantization matrices, we might uncover structural insights that translate to communication efficiency in computational tasks.', 'sentence_3': 'This conjecture connects the algebraic nature of boolean algebras with the practical aspects of computation, suggesting a novel approach to understanding complexity.'}

Taxonomy category: `SOS_DEGREE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fce36fe4b7a417f4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 25 out of 30 random seeds yield a Pearson correlation coefficient >= 0.7 between the minimal rank of the geometric quantization matrix

and communication protocol efficiency E[CP]. The conjecture is falsified if this condition is not met.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - geometric quantization AND boolean algebra AND communication complexity - quantization matrix rank AND boolean algebra AND communication protocol efficiency - Disjointness problem AND geometric quantization AND communication game efficiency

Top relevant hits considered: - [http://arxiv.org/abs/2601.23259v1] Geometric Quantization by Paths, Part III: The Metaplectic Anomaly - [http://arxiv.org/abs/math/0412257v7] PROP profile of deformation quantization and graph complexes with loops and wheels - [http://arxiv.org/abs/2306.09971v1] The R-matrix presentation for the rational form of a quantized enveloping algebra - [http://arxiv.org/abs/2512.03807v2] Algorithms for Boolean Matrix Factorization using Integer Programming and Heuristics - [http://arxiv.org/abs/1306.1114v1] On the complexity of Boolean matrix ranks - [http://arxiv.org/abs/0910.2033v1] A bound on the scrambling index of a primitive matrix using Boolean rank - [http://arxiv.org/abs/1002.1167v1] Geometric Programming Problem with Co-Efficients and Exponents Associated with Binary Numbers - [http://arxiv.org/abs/1102.4243v2] Disjointness and unique ergodicity of C*-dynamical systems

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_algebra(n):
        return [tuple(random.choice([0, 1]) for _ in range(n)) for _ in range(2**n)]

    def geometric_quantization_matrix(boolean_algebra):
        n = len(boolean_algebra[0])
        matrix = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                if all(x == y for x, y in zip(boolean_algebra[i], boolean_algebra[j])):
                    matrix[i][j] = 1
```

```
        return matrix

    def communication_protocol_efficiency(n):
        # Simplified model: O(log(n)) bits
        return math.log(n, 2)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        boolean_algebra = generate_boolean_algebra(n)
        matrix = geometric_quantization_matrix(boolean_algebra)
        rank = sum(1 for row in matrix if any(row))
        efficiency = communication_protocol_efficiency(n)
        results.append({"n": n, "rank": rank, "efficiency": efficiency})

    correlation_coefficient = 0
    if len(results) > 1:
        x_mean = sum(result["rank"] for result in results) / len(results)
        y_mean = sum(result["efficiency"] for result in results) / len(results)
        numerator = sum((result["rank"] - x_mean) * (result["efficiency"] - y_mean) for result in
        denominator = math.sqrt(sum((result["rank"] - x_mean)**2 for result in results)) * math.sq
        correlation_coefficient = numerator / denominator

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "conjecture_holds": correlation_coefficient >= 0.7,
        "counterexample": "" if correlation_coefficient >= 0.7 else "low_correlation"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='low_correlation' first_failing_seed={first_faili
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_data n_tested={len(results)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, making it impossible to evaluate the Pearson correlation coefficient and support fraction. | next: Re-run the experiment with a longer timeout or increase the number of seeds to ensure sufficient data collection.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15129
2	preregistration	ollama_remote	glm4:latest	0	0	9004
3	novelty	ollama_remote	glm4:latest	0	0	8162
4	novelty	ollama_remote	glm4:latest	0	0	9949
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12866
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7263
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7198
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9397
9	judge	ollama_remote	glm4:latest	0	0	15388

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 94356 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b7c1e9607c6e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b7c1e9607c6e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b7c1e9607c6e.tar.gz` (if generated)